

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Affiliated to Guru Gobind Singh Indraprastha University, Delhi

Department of Computer Science Engineering (Data Science)

MASTER PROJECT DOCUMENTATION

KarAI

Automated Tax Compliance and Anomaly Detection Platform

Document Type	Master Reference — Proposal + Tech Stack + SRS (IEEE 830) + SDD
Version	1.0 June 2026
Prepared By	Harsh Arora Nikhil Bokaria Himanshu Bhushan
Programme	B.Tech – CSE (Data Science), MAIT, GGSIPU
SRS Standard	IEEE Std 830-1998
Project Category	Minor Project — Software Based — Govt. Initiatives & National Mission
Status	Draft for Minor Project Submission

DOCUMENT GUIDE AND STRUCTURE

This document is the single authoritative reference for the KarAI project. It is structured into five parts, each serving a distinct purpose for development, review, and academic evaluation. All AI assistants, developers, and evaluators working on KarAI should use this document as the primary source of truth for all technical decisions, requirements, and design specifications.

Part	Title	Purpose	Audience
I	Project Proposal	Formal submission document covering problem statement, objectives, and outcomes	Evaluators, Faculty
II	Technology Stack — Deep Dive	Complete breakdown of every technology, version, role, and configuration used	AI Assistants, Developers
III	Software Requirements Spec.	IEEE 830 compliant specification of all functional and non-functional requirements	All Stakeholders
IV	Software Design Document	Architectural decisions, database schema, API spec, ML pipeline, deployment design	Developers, AI Assistants

Important Note for AI Assistants: This document is shared as a comprehensive context package. When assisting with KarAI development, treat the SRS as the source of WHAT to build, and the SDD as the source of HOW to build it. The tech stack section defines all permitted libraries and versions — do not suggest alternatives unless explicitly asked.

TABLE OF CONTENTS

Part I. — PROJECT PROPOSAL

1. Project Title
2. Problem Statement
3. Objectives of the Project
4. Technology Stack Overview
5. Domain / Area of Application
6. Expected Outcomes
7. SDG & National Mission Alignment

Part II. — TECHNOLOGY STACK — DEEP DIVE

1. Architecture Overview
2. Core Language (Python 3.10+)
3. ML & Data Science Stack
4. Backend & API Layer
5. Database & Storage
6. Frontend & Visualisation
7. Authentication & Security Libraries
8. Reporting & Export
9. DevOps & Deployment
10. Development & Experiment Tracking
11. Complete Dependency Matrix

Part III. — SOFTWARE REQUIREMENTS SPECIFICATION (IEEE 830)

1. Introduction
 2. Overall Description
 3. External Interface Requirements
 4. System Features & Functional Requirements
 5. Non-Functional Requirements
- Appendix A: Glossary
- Appendix B: Use Case Summary

Part IV. — SOFTWARE DESIGN DOCUMENT

1. Introduction & Design Goals
2. System Architecture Overview
3. Module & Component Design
4. Database Design (ER & Schema)
5. REST API Specification
6. ML Pipeline Architecture
7. Data Flow Design (DFD L0 & L1)
8. Security Architecture
9. Deployment Architecture
10. Error Handling Strategy

PART I

PROJECT PROPOSAL

KarAI — Automated Tax Compliance and Anomaly Detection Platform

PROJECT PROPOSAL — PROBLEM STATEMENT 1

KarAI

Automated Tax Compliance and Anomaly Detection Platform

Student Names	Harsh Arora Nikhil Bokaria Himanshu Bhushan
Programme	B.Tech – Computer Science Engineering (Data Science)
Project Type	Software Based Project
PS Source	Government Initiatives & National Mission (Digital India / GST Council of India)

1. PROJECT TITLE

KarAI — Automated Tax Compliance and Anomaly Detection Platform

2. PROBLEM STATEMENT

India's tax ecosystem — particularly GST and income tax compliance — poses significant operational challenges for Small and Medium Enterprises (SMEs), startups, and individual taxpayers. Manual review of financial records for compliance errors, anomalous transactions, and fraudulent patterns is time-consuming, error-prone, and requires specialised domain expertise that most small businesses cannot afford. Simultaneously, tax authorities face scalability issues in detecting non-compliance across millions of transactions filed daily.

There is a critical need for an intelligent, automated platform that can proactively identify compliance violations, flag statistically anomalous transactions using machine learning, and generate actionable audit reports — enabling businesses to self-assess and rectify issues before formal filing, while providing regulatory bodies with a scalable tool for risk-based scrutiny.

3. OBJECTIVES OF THE PROJECT

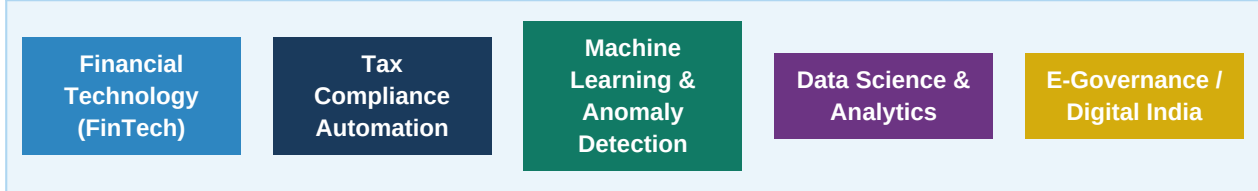
#	Objective
O1	Develop an ML-based anomaly detection engine using Isolation Forest and Random Forest algorithms to identify irregular or suspicious financial transactions in GST and income tax datasets.
O2	Build an automated compliance checker that validates uploaded transaction data against applicable GST rules and flags potential filing errors or regulatory breaches.

O3	Design an interactive risk-scoring dashboard (Power BI / Plotly) that visualises transaction health, anomaly clusters, and compliance gaps for business users.
O4	Implement a RESTful API backend (FastAPI) enabling seamless integration with third-party accounting and ERP software.
O5	Provide an automated report generation module that produces structured compliance summaries and anomaly logs suitable for CA/tax consultants.
O6	Evaluate system performance using precision, recall, F1-score and ROC-AUC on publicly available financial datasets.

4. TECHNOLOGY STACK OVERVIEW

Programming Language	Python 3.10+
ML / Data Science	Scikit-learn, XGBoost, LightGBM, SHAP, Pandas, NumPy, SciPy
Backend Framework	FastAPI, Pydantic, SQLAlchemy
Frontend / UI	React.js / Streamlit
Database	PostgreSQL
Visualisation	Plotly, Power BI, Matplotlib, Seaborn
Reporting	ReportLab, openpyxl
Dev Tools	Jupyter Notebook, MLflow, Git/GitHub, Docker, AWS EC2

5. DOMAIN / AREA OF APPLICATION



6. EXPECTED OUTCOMES

#	Expected Outcome
EO 1	A fully functional web application capable of ingesting financial transaction data (CSV/API) and detecting anomalies with a target accuracy of 85%+.
EO 2	An automated GST compliance report that identifies potential filing discrepancies and generates a downloadable audit summary.
EO 3	A risk-scored transaction dashboard highlighting high-risk entries with visual anomaly clustering for prioritised manual review.
EO 4	A REST API enabling third-party accounting tools (e.g., Tally, Zoho Books) to integrate with the anomaly detection engine.
EO 5	Estimated 60-70% reduction in manual compliance audit time for SME use cases.
EO 6	An IEEE-style technical report and paper draft documenting methodology, datasets, and experimental results.

7. SDG & NATIONAL MISSION ALIGNMENT

SDG 8	Decent Work and Economic Growth — Empowers MSMEs through affordable compliance automation, reducing overhead.
SDG 16	Peace, Justice and Strong Institutions — Promotes transparent, accountable tax practices at scale.
SDG 10	Reduced Inequalities — Levels the playing field for small businesses lacking dedicated finance/legal teams.
Digital India	Aligns with paperless governance, e-compliance, and digital financial infrastructure goals.
Startup India	Directly reduces regulatory burden on early-stage startups without dedicated compliance teams.

PART II

TECHNOLOGY STACK — DEEP DIVE

Complete reference for all libraries, versions, configurations, and design rationale

1. ARCHITECTURE OVERVIEW

KarAI is built on a **five-layer architecture**. Each layer has clearly defined responsibilities and communicates only with adjacent layers, ensuring separation of concerns and maintainability.

Layer	Name	Technologies	Responsibilities
L1	Presentation Layer	React.js, Streamlit, Plotly, Power BI	User interface, data visualisation, report download, dashboard rendering
L2	API Gateway Layer	FastAPI, Uvicorn, Pydantic, CORS Middleware	HTTP request routing, authentication validation, input schema enforcement, rate limiting
L3	Business Logic Layer	Python modules: Anomaly Engine, Compliance Validator, Report Generator	Core application logic — ML inference, rule validation, report composition
L4	Data Access Layer	SQLAlchemy ORM, Alembic (migrations)	Database abstraction, query execution, connection pooling, schema migrations
L5	Data Storage Layer	PostgreSQL 15+, File Storage (S3/local)	Persistent storage of users, transactions, results, reports, audit logs

Cross-Cutting Concerns

Concern	Implementation	Notes
Authentication	JWT (python-jose), bcrypt	All API routes except /auth/* require Bearer token. Tokens expire in 24h.
Logging	Python logging + structlog	Structured JSON logs. All actions and errors logged with request ID and user ID.
Monitoring	Prometheus + Grafana (production)	Tracks request latency, error rates, ML inference time, DB query time.
Caching	Redis (session data, rate limit counters)	Reduces DB load for repeated queries. TTL-based cache invalidation.
Configuration	.env files + Pydantic BaseSettings	All secrets and environment-specific config loaded from environment variables.
Testing	pytest, pytest-asyncio, httpx	Unit + integration tests. Target 70% coverage on business logic modules.

2. CORE LANGUAGE

Python 3.10+

Version	Python 3.10.x (minimum), 3.12.x recommended for production
Why Python	Dominant language for ML/DS ecosystem; extensive library support for Scikit-learn, FastAPI, Pandas; strong async capabilities in 3.10+
Key Features Used	Pattern matching (match/case), structural subtyping, union type hints (X Y), asyncio improvements
Environment	Virtual environment managed via conda or venv. All dependencies pinned in requirements.txt and requirements-dev.txt.
Style Standard	PEP 8. Enforced via flake8 with max-line-length=120. Type hints on all public functions (mypy strict mode recommended).
Version Manager	pyenv recommended for local development to manage multiple Python versions.

3. ML & DATA SCIENCE STACK

3.1 Scikit-learn 1.4+

Role	Primary ML library — Isolation Forest (unsupervised), Random Forest (supervised), preprocessing pipelines
Version	1.4.x (minimum)
Key Modules Used	sklearn.ensemble.IsolationForest, RandomForestClassifier; sklearn.preprocessing.StandardScaler, MinMaxScaler; sklearn.pipeline.Pipeline; sklearn.model_selection.cross_val_score; sklearn.metrics (precision, recall, f1, roc_auc)
Isolation Forest Config	n_estimators=200, max_samples="auto", contamination=0.05, max_features=1.0, bootstrap=False, random_state=42, warm_start=False
Random Forest Config	n_estimators=200, max_depth=None, min_samples_split=2, min_samples_leaf=1, random_state=42, class_weight="balanced" (for imbalanced fraud datasets)
Pipeline	All preprocessing steps wrapped in sklearn.pipeline.Pipeline to prevent data leakage during cross-validation and ensure reproducible inference.

3.2 XGBoost 2.0+

Role	Supervised anomaly classifier when labelled training data is available; used as ensemble member alongside Isolation Forest
Version	2.0.x (minimum)
Config	max_depth=6, learning_rate=0.1, n_estimators=200, subsample=0.8, colsample_bytree=0.8, use_label_encoder=False, eval_metric="logloss", random_state=42, scale_pos_weight=auto (handles class imbalance)
Early Stopping	early_stopping_rounds=20 on validation set to prevent overfitting. Best iteration stored and used for final inference.
Feature Importance	xgb.plot_importance() used for feature selection during model development. Weight, gain, and cover all tracked.

3.3 LightGBM 4.0+

Role	Alternative/ensemble supervised classifier; faster than XGBoost on larger datasets (>100K transactions)
Version	4.0.x (minimum)
Config	num_leaves=31, learning_rate=0.05, n_estimators=200, subsample=0.8, colsample_bytree=0.8, is_unbalance=True, random_state=42, verbose=-1
Advantage	Leaf-wise tree growth (faster) vs. XGBoost level-wise. Better on high-cardinality categorical features (party_name, category).
Categorical Handling	LightGBM native categorical feature support enabled for party_name and category columns. Avoids manual one-hot encoding.

3.4 SHAP 0.45+

Role	Model explainability — generates per-prediction feature importance values. Critical for regulatory transparency and user trust.
Version	0.45.x (minimum)
Explainer Type	shap.TreeExplainer — compatible with Isolation Forest, XGBoost, and LightGBM. $O(L * D)$ complexity where L=leaves, D=depth.
Config	TreeExplainer(model, check_additivity=False) for Isolation Forest (additivity not guaranteed for IF). check_additivity=True for supervised models.
Output	shap_values per transaction stored as JSONB in PostgreSQL. Top 5 contributing features extracted for dashboard display and report generation.
Visualisation	shap.waterfall_plot() rendered as base64 PNG per flagged transaction. shap.summary_plot() shown on model evaluation dashboard.
API Endpoint	GET /analysis/{run_id}/shap/{transaction_id} returns SHAP values and waterfall chart URL for a specific transaction.

3.5 Pandas 2.1+ and NumPy 1.26+

Pandas Role	Primary data manipulation library. Used for CSV ingestion, data cleaning, feature engineering, and result formatting.
Pandas Key Features	pd.read_csv() with dtype specification and parse_dates; GroupBy for party/category aggregations; merge() for ITC reconciliation; apply() for GSTIN validation; chunking for large datasets (chunksize=10000).
NumPy Role	Numerical computing backbone. Used for array operations, statistical calculations (z-scores, percentiles), and ML preprocessing.
NumPy Key Uses	np.log1p() for amount log-transform, np.percentile() for outlier thresholds, np.where() for vectorised conditional logic, np.isfinite() for data quality checks.

3.6 SciPy 1.12+

Role	Statistical analysis — z-scores, normality tests, correlation analysis
Key Modules	scipy.stats.zscore (amount standardisation), scipy.stats.shapiro (normality testing), scipy.spatial.distance (custom distance metrics for clustering visualisation)

4. BACKEND & API LAYER

4.1 FastAPI 0.111+

Role	Primary backend API framework. Handles all HTTP request routing, middleware, dependency injection.
Version	0.111.x (minimum)
Why FastAPI	Automatic OpenAPI docs generation, native async support (async/await), Pydantic integration for request/response validation, 3x faster than Flask in benchmarks.
Key Patterns Used	Dependency injection (Depends()), Background Tasks for async ML jobs, APIRouter for modular route organisation, HTTPException for standardised errors.
Middleware Stack	CORSMiddleware (whitelist-based), GZipMiddleware (compress responses > 1KB), TrustedHostMiddleware, custom RateLimitMiddleware (Redis-backed).
API Docs	Auto-generated Swagger UI at /docs and ReDoc at /redoc. Disabled in production (DOCS_ENABLED=false env var).
Server	Uvicorn with workers=4 (production). Gunicorn + Uvicorn workers for process-level parallelism.

4.2 Pydantic 2.5+

Role	Data validation and settings management. All request bodies, response schemas, and config are Pydantic models.
Version	2.5.x (minimum) — Pydantic v2 (significant performance improvements over v1)
Key Features	Field validators, model_validator for cross-field validation, ConfigDict for ORM mode, BaseSettings for environment config loading.
Config Loading	pydantic_settings.BaseSettings loads all env vars with type coercion. Fails fast on missing required config at startup.

4.3 SQLAlchemy 2.0+

Role	ORM for all database operations. Prevents SQL injection via parameterised queries. Manages connection pooling.
Version	2.0.x (minimum) — new async-first API
Async Mode	sqlalchemy.ext.asyncio with AsyncSession. All DB operations are async to avoid blocking the event loop.
Connection Pool	pool_size=10, max_overflow=20, pool_recycle=3600, pool_pre_ping=True (checks connections before use).
Migrations	Alembic for schema migrations. All schema changes versioned. Never run DDL in application code directly.

5. DATABASE & STORAGE

5.1 PostgreSQL 15+

Role	Primary relational database for all persistent data.
Version	15.x (minimum), 16.x recommended

Key Features	JSONB for SHAP values and raw transaction data; UUID primary keys (gen_random_uuid()); ENUM types for status fields; Partial indexes for performance; Row-level security for data isolation.
Indexes	B-Tree on user_id FK columns, date columns. GIN index on JSONB columns (raw_data, shap_values). Partial index on anomaly_results WHERE risk_category != "normal".
Connection	asyncpg driver with SQLAlchemy async. Connection string in DATABASE_URL env var.
Backups	pg_dump daily at 02:00 IST via cron. 7-day retention on S3.

5.2 Redis 7.0+

Role	In-memory store for: rate limit counters, JWT blacklist (logged-out tokens), session cache, job queue status.
Version	7.0.x (minimum)
Rate Limiting	redis-py with sliding window counter. Key: rate_limit:{user_id}:{endpoint}. Expiry: 60 seconds per window.
JWT Blacklist	On logout, JWT jti (JWT ID) stored in Redis with TTL = token expiry time. All requests check blacklist before processing.
Client	redis.asyncio.Redis (async client). Connection pool: max_connections=50.

6. FRONTEND & VISUALISATION

6.1 React.js 18+ (Primary Frontend)

Role	Primary web frontend for production deployment. Full-featured SPA with routing.
Key Libraries	React Router v6 (client-side routing), Axios (HTTP client), React Query v5 (server state management), Recharts (charting), shadcn/ui (component library).
State Management	React Query for server state. Zustand for lightweight client state (auth token, user preferences). No Redux (overkill for this scale).
Build Tool	Vite 5.x. Production build output to /dist with tree-shaking and code splitting.

6.2 Streamlit 1.35+ (Rapid Prototype / Admin Interface)

Role	Admin dashboard and rapid prototype interface. Used for model evaluation dashboard and admin controls.
Key Features	st.plotly_chart() for interactive charts, st.dataframe() with sorting/filtering, st.file_uploader() for CSV upload, st.download_button() for report download.
Session State	st.session_state for maintaining login state, selected dataset, and filter parameters across reruns.

6.3 Plotly 5.20+ and Power BI

Plotly Role	Interactive charts in both React (plotly.js) and Streamlit (plotly.express). Scatter plots, time-series, heatmaps, bar charts.
Key Chart Types	Scatter (anomaly score vs. amount), Time-series (compliance score trend), Bar (violation categories), Heatmap (party x category anomaly matrix).

Power BI	Optional embedded dashboard for enterprise users via Power BI REST API. Compliance score KPIs and drill-down by filing period.
Matplotlib/Seaborn	Used in Jupyter notebooks during model development and in PDF report generation (chart images embedded in ReportLab output).

7. AUTHENTICATION & SECURITY LIBRARIES

7.1 python-jose 3.3+ (JWT)

Role	JWT token generation, signing, and verification.
Algorithm	HS256 (HMAC-SHA256) for development. RS256 (RSA-SHA256) recommended for production (asymmetric keys allow microservice verification without secret sharing).
Payload	sub (user_id), role, jti (unique token ID for blacklisting), exp (expiry), iat (issued at).
Secret Key	Loaded from JWT_SECRET_KEY env var. Minimum 256-bit random key. Generated via: openssl rand -hex 32.

7.2 bcrypt / passlib

Role	Password hashing and verification.
Library	passlib[bcrypt] — passlib provides a unified interface; bcrypt is the underlying hashing algorithm.
Config	CryptContext(schemes=["bcrypt"], deprecated="auto"). Cost factor: 12 (default). Automatically upgrades to new schemes.
Policy	Passwords never stored or logged. Comparison always via constant-time hash.verify() to prevent timing attacks.

7.3 python-cryptography 42+

Role	AES-256 encryption for financial data at rest. TLS certificate management.
Key Usage	cryptography.fernet.Fernet for symmetric encryption of sensitive fields before storage. ENCRYPTION_KEY env var holds the Fernet key.
Fields Encrypted	amount, party_name, gstin in the transactions table. Decrypted only in the business logic layer, never at the ORM level.

8. REPORTING & EXPORT

8.1 ReportLab 4.1+

Role	PDF generation for compliance audit reports. Produces structured, branded PDFs with tables, charts, and narrative.
Version	4.1.x (minimum)
Key Modules	platypus (high-level layout), pdfgen.canvas (low-level drawing), lib.styles (text styles), lib.units (mm, cm for positioning).
Chart Embedding	Plotly charts exported as base64 PNG via plotly.io.to_image() and embedded in PDF via reportlab.platypus.Image.

8.2 openpyxl 3.1+

Role	Excel (.xlsx) export of full anomaly results with formatting. Color-coded anomaly score cells.
Key Features	Conditional formatting (red-yellow-green scale on anomaly_score column), auto-filter on all columns, freeze top row, number formatting for amounts.

9. DEVOPS & DEPLOYMENT

9.1 Docker & Docker Compose 24+

Role	Containerisation for consistent deployment across environments.
Services	5 containers: karai-api (FastAPI), karai-frontend (React/Nginx), karai-db (PostgreSQL), karai-redis (Redis), karai-worker (background ML jobs via Celery)
Base Image	python:3.12-slim for API and worker. node:20-alpine for frontend build. postgres:16-alpine for database.
Multi-stage Build	Frontend: node:20-alpine (build stage) → nginx:alpine (serve stage). Reduces final image size from ~800MB to ~25MB.
Health Checks	All services have HEALTHCHECK defined. docker-compose depends_on with condition: service_healthy.
Volumes	Named volume karai-pgdata for PostgreSQL persistence. Bind mount for .env files.

9.2 AWS Deployment Topology

API Server	EC2 t3.large (2 vCPU, 8GB RAM). Runs Docker Compose with API + worker containers.
Database	RDS PostgreSQL db.t3.medium (2 vCPU, 4GB RAM). Multi-AZ for production. Automated backups enabled.
File Storage	S3 bucket for generated PDF/Excel reports and uploaded CSVs. Lifecycle policy: delete after 90 days.
Load Balancer	ALB (Application Load Balancer) with HTTPS listener. SSL certificate via AWS Certificate Manager.
CDN	CloudFront distribution for React frontend static assets. Cache-Control headers set for versioned assets.
DNS	Route 53 for domain management. Health check failover routing.

9.3 CI/CD — GitHub Actions

Trigger	Push to main branch or PR to main.
Pipeline Steps	1. Checkout code → 2. Run pytest → 3. Run flake8 → 4. Build Docker images → 5. Push to ECR → 6. Deploy to EC2 via SSH + docker-compose pull + up
Secrets	All credentials (AWS keys, DB URL, JWT secret) stored in GitHub Secrets. Never in code or Dockerfile.
Branch Strategy	main (production), develop (integration), feature/* (individual features). PRs require 1 review + CI pass.

10. DEVELOPMENT & EXPERIMENT TRACKING

10.1 MLflow 2.13+

Role	ML experiment tracking, model registry, and artifact storage.
Tracking	mlflow.start_run() wraps every model training. Logs: parameters (contamination, n_estimators), metrics (precision, recall, F1, AUC-ROC), artifacts (model.pkl, feature importance chart).
Model Registry	Trained models registered via mlflow.sklearn.log_model(). Production model loaded via mlflow.sklearn.load_model() at API startup.
UI	MLflow UI served at :5000 (development only). Accessible to team for comparing experiments.

10.2 Other Development Tools

Tool	Version	Purpose
Jupyter Notebook	7.1+	Model development, EDA, feature engineering prototyping
pytest	8.0+	Unit and integration testing. Fixtures in conftest.py. Async tests via pytest-asyncio.
httpx	0.27+	Async HTTP client for testing FastAPI endpoints in pytest.
flake8	7.0+	PEP 8 linting. max-line-length=120. Plugins: flake8-bugbear, flake8-import-order.
pre-commit	3.7+	Git hook runner. Runs flake8 + black + isort before every commit.
black	24.0+	Code formatter. line-length=120. Enforces consistent style.
isort	5.13+	Import sorter. Compatible with black. profile="black".
Postman	v11	API testing and documentation during development.
pgAdmin 4	v8	PostgreSQL GUI for schema inspection and manual query execution.
Git	2.40+	Version control. Branching strategy: main/develop/feature/* as described above.

11. COMPLETE DEPENDENCY MATRIX

The following is the authoritative requirements.txt for KarAI. All AI assistants should use these exact packages when generating code, suggesting imports, or setting up the environment.

[illegible]

PART III

SOFTWARE REQUIREMENTS SPECIFICATION

IEEE Std 830-1998 Compliant — KarAI v1.0

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for KarAI — an Automated Tax Compliance and Anomaly Detection Platform. It serves as the primary reference document for the development team, academic supervisors, and project evaluators involved in the design, implementation, and assessment of the system during the Minor Project phase, in accordance with IEEE Std 830-1998.

1.2 Project Scope

KarAI is a web-based intelligent platform designed to assist SMEs, individual taxpayers, and tax consultants in proactively managing tax compliance through ML-powered anomaly detection and automated audit reporting. It is NOT a replacement for existing accounting software but operates as a complementary analytical layer.

In Scope:

- ML-based anomaly detection on financial transaction datasets
- GST compliance rule validation and scoring
- Interactive risk-scoring dashboard
- Automated PDF and Excel audit report generation
- REST API for third-party accounting tool integration
- Role-based user authentication and data isolation

Out of Scope (v1.0):

- Direct GST return filing to the GSTN portal
- Real-time transaction monitoring (batch processing only)
- Full ERP integration beyond REST API
- Multilingual interface (English only)

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
GST	Goods and Services Tax — India's unified indirect tax system
GSTIN	15-digit alphanumeric GST Identification Number assigned to registered businesses
ITC	Input Tax Credit — credit mechanism within GST for offsetting tax on inputs

HSN	Harmonised System of Nomenclature — commodity classification code used in GST
ML	Machine Learning
SHAP	SHapley Additive Explanations — model explainability framework
Isolation Forest	Unsupervised ML algorithm that isolates anomalies using random decision trees
Anomaly Score	Continuous value 0.0-1.0 representing deviation from normal transaction patterns
FR	Functional Requirement
NFR	Non-Functional Requirement
JWT	JSON Web Token — stateless authentication standard
ORM	Object-Relational Mapping (SQLAlchemy)
SRS	Software Requirements Specification

1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- GST Council of India — Official Filing Guidelines (gstn.org.in)
- Income Tax Act, 1961 (as amended) — Ministry of Finance, Government of India
- Scikit-learn Documentation v1.4 — scikit-learn.org
- FastAPI Documentation v0.111 — fastapi.tiangolo.com
- OWASP Top 10 Web Application Security Risks (2023)

2. OVERALL DESCRIPTION

2.1 Product Perspective

KarAI is a standalone web application serving as a complementary compliance intelligence layer for Indian businesses. It consists of three primary architectural components: (1) ML Backend in Python/FastAPI handling inference, SHAP, and reports; (2) React.js/Streamlit Frontend providing dashboard, upload, and report UI; (3) PostgreSQL Database storing all transaction records, results, users, and audit logs.

2.2 User Classes and Characteristics

User Class	Technical Level	Primary Actions
SME Owner / Finance Manager	Low — basic computer literacy	Upload data, view compliance score, download reports
Tax Consultant / CA	Moderate — familiar with tax tools	Review flagged transactions, validate anomaly flags, generate client reports
System Administrator	High — technical background	Manage users, configure ML thresholds, monitor system health, retrain models

2.3 Operating Environment

Server OS	Ubuntu 22.04 LTS or Docker-compatible Linux
Backend Runtime	Python 3.12+, FastAPI/Uvicorn
Database	PostgreSQL 15+, Redis 7+

Frontend	React.js (Node.js 20+) or Streamlit 1.35+
Browser Support	Chrome 90+, Firefox 88+, Edge 90+
Deployment Target	AWS EC2 t3.large / Docker / Render
Min Client HW	Any device with modern browser and stable internet connection

2.4 Design and Implementation Constraints

- System must comply with applicable provisions of the Indian GST Act and CBDT regulations for FY 2025-26.
- All sensitive financial data must be encrypted using AES-256 at rest and TLS 1.3 in transit.
- ML models must provide SHAP-based explanations for every flagged transaction (regulatory transparency).
- System must process datasets up to 100,000 transactions without timeout or data loss.
- No user financial data may be shared with third parties without explicit consent.
- Codebase must follow PEP 8 (Python) and ESLint (JavaScript). All ML models must be versioned via MLflow.

3. EXTERNAL INTERFACE REQUIREMENTS

3.1 User Interface Screens

- Login/Registration: Email + password auth. JWT token issued on login. Forgot Password via email OTP.
- Dashboard Home: KPI cards — Total Transactions, Flagged Count, Compliance Score (0-100), Risk Level.
- Data Upload: Drag-and-drop CSV up to 50MB. Field mapping wizard for non-standard schemas.
- Anomaly Results: Paginated table with anomaly scores, risk badges, expandable SHAP charts per row.
- Compliance Report: Violation list by severity, compliance score trend chart, remediation suggestions.
- Report Download: One-click PDF/Excel generation. Report history with 90-day retention.
- Settings: Anomaly sensitivity threshold, notification preferences, API key management.

3.2 Software Interfaces

PostgreSQL 15+	Primary database via SQLAlchemy async ORM with Pydantic schema validation
Redis 7+	Rate limiting, JWT blacklist, session cache
Scikit-learn 1.4+	Isolation Forest and Random Forest implementations
XGBoost / LightGBM	Supervised anomaly classification
SHAP 0.45+	Per-prediction feature importance
FastAPI 0.111+	REST API with auto-generated OpenAPI docs
Tally / Zoho Books	Optional integration via REST API for transaction data pull
ReportLab / openpyxl	PDF and Excel audit report generation

3.3 Communications Interfaces

- HTTPS (TLS 1.3): All client-server communication encrypted end-to-end.
- REST API (JSON): Internal and external integrations follow RESTful conventions.
- WebSocket: Real-time processing status updates during ML inference on large datasets.
- SMTP: Transactional emails for OTP, password reset, and report-ready notifications.

4. SYSTEM FEATURES AND FUNCTIONAL REQUIREMENTS

Functional requirements are organised by module. Each has a unique identifier (FR-XX), description, and priority (High/Medium/Low).

4.1 User Authentication and Role Management

Req. ID	Description	Priority
FR-01	Provide email and password-based user registration with input validation (email format, minimum 8-character alphanumeric password).	High
FR-02	Authenticate users using JWT tokens with configurable expiry (default 24h). Tokens invalidated on logout via Redis blacklist.	High
FR-03	Support three roles: Administrator, Tax Consultant, SME User — each with defined feature permissions enforced at the API level.	High
FR-04	Enforce row-level data isolation — SME Users can only access their own uploaded datasets, results, and reports.	High
FR-05	Provide password reset via email OTP with 10-minute expiry window.	Medium
FR-06	Lock accounts after 5 consecutive failed login attempts and notify user via email.	Medium

4.2 Transaction Data Ingestion

Req. ID	Description	Priority
FR-07	Accept CSV files up to 50MB via drag-and-drop upload interface.	High
FR-08	Validate CSV schema requiring: transaction_id, date, amount, party_name, gstin, category columns.	High
FR-09	Provide field mapping wizard for non-standard CSV formats.	High
FR-10	Store all ingested transaction data in PostgreSQL with user-level isolation.	High
FR-11	Provide REST API endpoints for programmatic data ingestion from third-party accounting tools.	Medium
FR-12	Reject files exceeding 500,000 rows with guidance to use the API for large datasets.	Medium
FR-13	Display real-time upload progress bar and notify user on completion or failure.	Low

4.3 Anomaly Detection Engine

Req. ID	Description	Priority
FR-14	Apply Isolation Forest as primary unsupervised anomaly detection algorithm (contamination=0.05, configurable).	High
FR-15	Apply XGBoost or LightGBM for supervised classification when user-labelled training data is available.	High
FR-16	Assign continuous anomaly score (0.0-1.0) to each transaction.	High
FR-17	Classify each transaction into: Normal (0-0.3), Suspicious (0.3-0.6), High-Risk (0.6-0.85), Critical (0.85-1.0).	High

FR-18	Generate SHAP feature importance values for every flagged transaction.	High
FR-19	Allow administrators to adjust anomaly detection sensitivity threshold via Settings without retraining.	Medium
FR-20	Log all model inference runs including version, parameters, dataset hash, and timestamp.	Medium
FR-21	Support model retraining on user-provided labelled datasets via admin interface.	Low

4.4 GST Compliance Validator

Req. ID	Description	Priority
FR-22	Validate format of all GSTIN values against the 15-character alphanumeric standard.	High
FR-23	Detect duplicate invoice entries (same invoice number, date, and GSTIN) within a filing period.	High
FR-24	Flag transactions exceeding GST threshold limits that are missing required HSN/SAC codes.	High
FR-25	Identify potential ITC mismatches by comparing purchase-side and supply-side transaction entries.	High
FR-26	Generate compliance score (0-100) per filing period based on ratio and severity of violations.	High
FR-27	Categorise violations by severity (Critical, Major, Minor) with descriptions and remediation steps.	Medium
FR-28	Maintain historical compliance score records across filing periods for trend analysis.	Low

4.5 Risk Dashboard

Req. ID	Description	Priority
FR-29	Display KPI cards: Total Transactions Processed, Flagged Count, Compliance Score, Overall Risk Level.	High
FR-30	Render time-series chart of anomaly counts and compliance scores across selected date range.	High
FR-31	Display scatter plot of transactions by anomaly score vs. amount to visualise anomaly clusters.	High
FR-32	Provide filterable, sortable, paginated transaction table with anomaly score and risk category badges.	High
FR-33	Display SHAP waterfall charts in expandable row detail view for each flagged transaction.	High
FR-34	Support export of dashboard view as PNG/PDF via dedicated export button.	Medium
FR-35	Support date range and category filtering across all dashboard visualisations.	Medium

4.6 Automated Report Generation

Req. ID	Description	Priority
FR-36	Generate structured PDF audit report including executive summary, compliance score, flagged transactions, SHAP charts, and remediation recommendations.	High
FR-37	Generate Excel export of all flagged transactions with anomaly scores, SHAP feature values, and risk categories.	High

FR-38	Include in all reports: user details, filing period, generation timestamp, dataset hash, and model version metadata.	High
FR-39	Maintain report history in user dashboard with download capability for 90 days.	Medium
FR-40	Allow Tax Consultants to add manual annotations to flagged items before generating client-facing report.	Low

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance

Req. ID	Description	Priority
NFR-01	Process 10,000 transactions through full ML pipeline (ingestion, inference, SHAP, scoring) within 30 seconds.	High
NFR-02	Dashboard home page loads within 3 seconds on 10 Mbps+ connection.	High
NFR-03	All non-ML REST API endpoints respond within 500ms under normal load.	High
NFR-04	Support minimum 50 concurrent users without measurable performance degradation (< 20% response time increase).	Medium
NFR-05	PDF report generation for 10,000 transactions completes within 15 seconds.	Medium

5.2 Security

Req. ID	Description	Priority
NFR-06	Hash passwords using bcrypt with minimum cost factor 12. No plaintext passwords stored or logged.	High
NFR-07	All API endpoints except /auth/* require valid JWT Bearer token. Unauthorised requests return HTTP 401.	High
NFR-08	Financial data stored in DB encrypted at rest using AES-256 (Fernet). Keys stored separately.	High
NFR-09	Implement rate limiting: 100 requests/minute per user. Exceeding returns HTTP 429.	High
NFR-10	All DB queries use parameterised statements via SQLAlchemy ORM. No raw SQL string concatenation.	High
NFR-11	Sanitise all user inputs server-side to prevent XSS and SQL injection.	High
NFR-12	Implement CORS policies restricting API to approved frontend origins only.	Medium

5.3 Reliability & Availability

Req. ID	Description	Priority
NFR-13	Target 99% uptime during business hours (09:00-21:00 IST, Monday-Saturday).	High
NFR-14	Automated daily database backups at 02:00 IST with 7-day retention.	High
NFR-15	System recovers to last consistent state within 5 minutes of failure.	Medium
NFR-16	ML inference failures handled gracefully. Failed jobs queued for retry up to 3 times.	Medium

5.4 Maintainability

Req. ID	Description	Priority
NFR-17	All Python code conforms to PEP 8. JavaScript passes ESLint.	High
NFR-18	All ML models version-controlled via MLflow. Each version records parameters, metrics, dataset hash.	High
NFR-19	Entire application containerised via Docker. One docker-compose up command for local deployment.	Medium
NFR-20	All API endpoints documented in auto-generated OpenAPI (Swagger) at /docs.	Medium
NFR-21	Unit test coverage minimum 70% for all backend modules.	Low

5.5 Usability

Req. ID	Description	Priority
NFR-22	New user registers, uploads data, and views anomaly results within 5 minutes without formal training.	High
NFR-23	All SHAP charts accompanied by plain-language tooltip explanations for non-technical users.	High
NFR-24	UI fully responsive on desktop (1920x1080) and tablet (768x1024).	Medium
NFR-25	Clear actionable error messages for all failure scenarios (invalid file, network error, model timeout).	Medium

5.6 Portability

Req. ID	Description	Priority
NFR-26	Deployable on any Docker-compatible host (AWS EC2, GCP, Render) without code modifications.	High
NFR-27	Supports PostgreSQL 14+ without schema or query modifications.	Medium
NFR-28	All environment-specific config (DB URLs, API keys, thresholds) managed via .env files. Never hardcoded.	High

APP A. GLOSSARY

Term	Definition
Anomaly Score	Continuous value 0.0-1.0 from ML model indicating statistical deviation. Higher = more suspicious.
Compliance Score	Percentage 0-100 per filing period based on proportion and severity of detected GST violations.
Contamination	Isolation Forest hyperparameter: expected proportion of anomalies in dataset (default 0.05 = 5%).
Feature Importance	SHAP-derived values showing each feature's contribution to a specific anomaly prediction.
GSTIN	15-digit alphanumeric tax identification number for GST-registered entities in India.
ITC Mismatch	Discrepancy between input tax credit claimed by buyer and tax declared by supplier in GST returns.
Isolation Forest	Unsupervised ML algorithm that detects anomalies by isolating observations using random decision trees.

SHAP	SHapley Additive Explanations — game-theoretic framework for explaining individual ML predictions.
------	--

APP B. USE CASE SUMMARY

UC-01: User Registration and Login

Use Case ID	UC-01
Name	User Registration and Login
Primary Actor	All User Classes
Preconditions	User has valid email.
Main Flow	1. User navigates to registration. 2. Enters name, email, password, role. 3. System validates and creates account. 4. Login → JWT token issued. 5. Redirected to role-appropriate dashboard.
Postconditions	User authenticated with access to role-appropriate features.
Exceptions	Email exists → error. Invalid credentials → error + lockout counter increment.

UC-02: Upload Transaction Data

Use Case ID	UC-02
Name	Upload Transaction Data
Primary Actor	SME User / Tax Consultant
Preconditions	User authenticated. CSV data available.
Main Flow	1. Navigate to Upload. 2. Drag-drop CSV. 3. System validates size and schema. 4. Field mapping wizard if needed. 5. Data stored in PostgreSQL.
Postconditions	Dataset stored and ready for analysis.
Exceptions	File > 50MB → reject. Invalid schema → prompt mapping or reject.

UC-03: Run Anomaly Detection

Use Case ID	UC-03
Name	Run Anomaly Detection
Primary Actor	SME User / Tax Consultant
Preconditions	At least one ingested dataset exists.
Main Flow	1. Select dataset → click Analyse. 2. Isolation Forest + optional XGBoost run. 3. SHAP computed for flagged transactions. 4. Results stored. 5. User redirected to Results view.
Postconditions	All transactions have anomaly scores, risk classifications, and SHAP explanations.
Exceptions	Large dataset → job queued + email notification on completion.

UC-04: View GST Compliance Report

Use Case ID	UC-04
-------------	-------

Name	View GST Compliance Report
Primary Actor	SME User / Tax Consultant
Preconditions	Anomaly detection completed.
Main Flow	1. Navigate to Compliance tab. 2. System shows score, violation categories, flagged entries. 3. User reviews with remediation steps. 4. Optionally downloads report.
Postconditions	User understands violations and remediation steps.
Exceptions	No GST data → informational message.

UC-05: Generate Audit Report

Use Case ID	UC-05
Name	Generate Audit Report
Primary Actor	SME User / Tax Consultant
Preconditions	Analysis and compliance validation completed.
Main Flow	1. Navigate to Reports. 2. Select PDF or Excel. 3. Select filing period and dataset. 4. System generates report. 5. File downloaded.
Postconditions	Downloadable audit report generated and in report history.
Exceptions	Generation timeout → retry queued; user notified.

PART IV

SOFTWARE DESIGN DOCUMENT

Architecture, Database Schema, API Spec, ML Pipeline, Deployment

1. INTRODUCTION & DESIGN GOALS

Document Purpose	Defines HOW KarAI is built. Translates SRS requirements into concrete architectural and implementation decisions.
Design Philosophy	Separation of concerns, fail-fast validation, security-by-default, explainable AI, stateless API layer.
Primary Readers	Developers, AI assistants, and technical reviewers building or extending the KarAI codebase.

Design Goals

Goal	Description	Realised By
Modularity	Each functional area is an independent module with clear interfaces	Python modules per feature; APIRouter per domain
Explainability	Every ML prediction must be traceable to feature contributions	SHAP TreeExplainer on all models; SHAP values stored in DB
Security First	Security decisions are architectural, not afterthoughts	JWT + bcrypt + AES-256 + ORM + CORS baked into base classes
Async First	No blocking I/O in the request-response cycle	SQLAlchemy async, asyncpg, Celery workers for ML jobs
Reproducibility	All ML experiments must be reproducible	MLflow + random_state=42 + dataset hash tracking
Observability	System state visible at all times	Structured logging + Prometheus metrics + health endpoints

2. SYSTEM ARCHITECTURE OVERVIEW

2.1 Layered Architecture

Layer	Components	Communication	Design Pattern
Presentation	React.js SPA, Streamlit Admin UI	HTTPS → REST API, WebSocket for streaming	MVC (View layer)
API Gateway	FastAPI routers, Pydantic validators, JWT middleware, CORS	Receives HTTP requests, forwards to services	Facade + Middleware Chain

Business Logic	AnomalyEngine, ComplianceValidator, ReportGenerator, AuthService	Pure Python function calls, no DB access	Service Layer + Strategy Pattern
Data Access	SQLAlchemy ORM models, Repository classes, Alembic migrations	Async SQL queries to PostgreSQL	Repository Pattern
Infrastructure	PostgreSQL, Redis, S3 (reports), SMTP	asyncpg (DB), redis-py (cache), boto3 (S3)	Adapter Pattern

2.2 Component Interaction Map

Source Component	Calls	Via	Returns
React Frontend	FastAPI Router	HTTPS REST	JSON Response
FastAPI Router	AuthService	Dependency injection	User object or 401
FastAPI Router	AnomalyEngine	Function call	Celery task ID
Celery Worker	AnomalyEngine.run()	Task queue (Redis)	Stores results in DB
AnomalyEngine	FeaturePipeline	Function call	Feature DataFrame
AnomalyEngine	IsolationForest / XGB	scikit-learn API	Scores array
AnomalyEngine	SHAPExplainer	shap.TreeExplainer	shap_values dict
AnomalyEngine	TransactionRepository	ORM async call	Stored results
ComplianceValidator	RuleEngine	Function call	Violation list
ReportGenerator	TransactionRepository	ORM async call	Transaction data
ReportGenerator	ReportLab / openpyxl	Library call	PDF/Excel bytes

3. MODULE & COMPONENT DESIGN

3.1 AuthService

File	app/services/auth.py
Responsibilities	User registration, login, JWT token generation/verification, password hashing, account logout.
Key Functions	register_user(email, password, role) → User hash_password(plain) → str create_token(user_id, role) → str verify_token(token) → TokenPayload increment_failed_attempts(user_id)
Dependencies	UserRepository, bcrypt (passlib), python-jose, Redis (JWT blacklist)
Error Handling	Raises HTTPException(401) on invalid token. HTTPException(403) on locked account. HTTPException(409) on duplicate email.

3.2 DataIngestionService

File	app/services/ingestion.py
Responsibilities	CSV parsing, schema validation, field mapping, chunked storage to PostgreSQL.

Key Functions	<code>validate_csv(file) → ValidationResult</code> <code>map_fields(df, mapping) → DataFrame</code> <code>store_transactions(df, dataset_id) → int</code> (row count)
Chunking	Reads CSV in chunks of 10,000 rows using <code>pd.read_csv(chunksize=10000)</code> . Each chunk inserted via bulk ORM insert.
Dependencies	Pandas, DatasetRepository, TransactionRepository, Pydantic validators
Validation Rules	Required columns check, date format validation, amount positive check, GSTIN format pre-check, duplicate detection within upload batch.

3.3 FeatureEngineeringPipeline

File	<code>app/ml/features.py</code>
Responsibilities	Transforms raw transaction data into ML-ready feature matrix. Wrapped in sklearn Pipeline for reproducibility.
Output Features	16 features per transaction (see Section 6 for full list)
Pipeline Steps	1. DateEncoder (day_of_week, month, quarter, is_weekend) → 2. AmountTransformer (log1p, zscore) → 3. PartyFrequencyEncoder (party_count, party_amount_mean, party_amount_std) → 4. CategoryEncoder → 5. GSTINValidator → 6. DuplicateFlagger → 7. StandardScaler
Fit/Transform	<code>Pipeline.fit()</code> on training data. <code>Pipeline.transform()</code> on inference data. Pipeline serialised via joblib alongside model in MLflow artifact.

3.4 AnomalyDetectionEngine

File	<code>app/ml/engine.py</code>
Responsibilities	Orchestrates full ML inference pipeline: load model → run feature pipeline → run inference → compute SHAP → classify risk → store results.
Key Functions	<code>run_detection(dataset_id, model_run_id) → ModelRunResult</code> <code>classify_risk(score) → RiskCategory</code> <code>load_model(run_id) → Pipeline</code>
Celery Task	@ <code>celery_app.task(bind=True)</code> decorator. Task ID returned to API immediately. Results pushed to DB asynchronously. WebSocket broadcasts status updates.
Model Loading	Production model URI loaded from <code>MLFLOW_MODEL_URI</code> env var. Model cached in memory after first load. Reloaded on version change signal.
SHAP	<code>shap.TreeExplainer(model[-1])</code> — uses last step of sklearn Pipeline (the estimator). <code>top_5_features</code> extracted from <code>shap_values</code> for quick display.

3.5 ComplianceValidator

File	<code>app/services/compliance.py</code>
Responsibilities	Rule-based GST compliance checking. Produces violation list and compliance score.

Rules Implemented	RULE-001: GSTIN format validation (regex: <code>^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]\$</code>) RULE-002: Duplicate invoice detection (hash of invoice_no + date + gstin) RULE-003: HSN code missing for B2B transactions > Rs 50,000 RULE-004: ITC eligibility check (ineligible categories flagged) RULE-005: Round-trip transaction detection RULE-006: Threshold anomalies (unusually high ITC claim vs turnover)
Scoring Formula	$\text{compliance_score} = 100 - (\text{critical_count} * 10 + \text{major_count} * 5 + \text{minor_count} * 2)$. Floored at 0.

3.6 ReportGenerator

File	app/services/reports.py
Responsibilities	Generates formatted PDF (ReportLab) and Excel (openpyxl) audit reports.
PDF Sections	1. Cover Page 2. Executive Summary 3. Compliance Score Card 4. Top Anomalies Table 5. Violation Detail (by severity) 6. SHAP Charts (embedded PNG) 7. Remediation Checklist 8. Appendix: Full Transaction List
Excel Sheets	Sheet 1: Summary KPIs Sheet 2: All Transactions with anomaly scores Sheet 3: Violations Sheet 4: SHAP Values
Storage	Generated file saved to S3 (production) or /reports/ (local). Path stored in reports table. Presigned S3 URL returned to frontend for download.
Async	Report generation runs as Celery task. Frontend polls GET /reports/{id}/status. On completion, download URL available.

4. DATABASE DESIGN

4.1 Entity-Relationship Overview

Entity	Relates To	Relationship	Cardinality
users	datasets	A user owns many datasets	1 : N
users	reports	A user generates many reports	1 : N
datasets	transactions	A dataset contains many transactions	1 : N
datasets	model_runs	A dataset can have many analysis runs	1 : N
datasets	compliance_checks	A dataset can have many compliance checks	1 : N
model_runs	anomaly_results	A model run produces one result per transaction	1 : N
transactions	anomaly_results	A transaction has results from multiple model runs	1 : N
compliance_checks	violations	A compliance check produces many violations	1 : N
transactions	violations	A transaction can have multiple violations	1 : N
users	audit_logs	All user actions produce audit log entries	1 : N

4.2 Table Schemas

Table: users

Column	Data Type	Constraint	Description
id	UUID	PK, NOT NULL, DEFAULT gen_random_uuid()	Auto-generated unique user identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	User email address — login credential
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash (cost=12). Never stored in plaintext.
role	VARCHAR(20)	NOT NULL, CHECK IN (admin, consultant, sme_user)	User role controlling feature access
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Account enabled/disabled flag
failed_attempts	INTEGER	NOT NULL, DEFAULT 0	Consecutive failed login counter. Reset on success.
locked_until	TIMESTAMP	NULL	Account lockout expiry. NULL = not locked.
created_at	TIMESTAMP	NOT NULL, DEFAULT now()	Account creation timestamp (UTC)
updated_at	TIMESTAMP	NOT NULL, DEFAULT now()	Last update timestamp (UTC). Auto-updated via trigger.

Table: datasets

Column	Data Type	Constraint	Description
id	UUID	PK, NOT NULL	Unique dataset identifier
user_id	UUID	FK(users.id) ON DELETE CASCADE, NOT NULL	Owning user
filename	VARCHAR(255)	NOT NULL	Original uploaded filename
row_count	INTEGER	NOT NULL	Number of transactions in dataset
file_hash	VARCHAR(64)	NOT NULL	SHA-256 hash of original file for deduplication
status	VARCHAR(20)	NOT NULL, DEFAULT pending	pending / processing / ready / error
filing_period	VARCHAR(20)	NULL	e.g. 2025-Q4 or 2025-10 (month). User-specified.
upload_date	TIMESTAMP	NOT NULL, DEFAULT now()	Upload timestamp (UTC)

Table: transactions

Column	Data Type	Constraint	Description
id	UUID	PK, NOT NULL	Unique transaction identifier
dataset_id	UUID	FK(datasets.id) ON DELETE CASCADE, NOT NULL	Parent dataset
transaction_id	VARCHAR(100)	NOT NULL	Original transaction ID from source CSV
txn_date	DATE	NOT NULL	Transaction date

amount	NUMERIC(15,2)	NOT NULL	Transaction amount (stored encrypted in production)
party_name	VARCHAR(255)	NOT NULL	Counter-party name (encrypted)
gstn	VARCHAR(15)	NULL	Party GSTIN (encrypted). NULL if not applicable.
category	VARCHAR(100)	NULL	HSN/SAC code or user-defined category
invoice_number	VARCHAR(100)	NULL	Invoice/bill number for GST compliance checks
raw_data	JSONB	NOT NULL	Complete original CSV row stored as JSON for audit

Table: model_runs

Column	Data Type	Constraint	Description
id	UUID	PK, NOT NULL	Unique analysis run identifier
dataset_id	UUID	FK(datasets.id), NOT NULL	Target dataset for this run
model_name	VARCHAR(50)	NOT NULL	isolation_forest / xgboost / lightgbm / ensemble
model_version	VARCHAR(20)	NOT NULL	MLflow run ID of the model used
parameters	JSONB	NOT NULL	Hyperparameters used: {contamination, n_estimators, ...}
metrics	JSONB	NULL	Performance metrics if labelled data available: {precision, recall, f1, auc}
status	VARCHAR(20)	NOT NULL, DEFAULT running	running / completed / failed
run_timestamp	TIMESTAMP	NOT NULL, DEFAULT now()	When inference started (UTC)
completed_at	TIMESTAMP	NULL	When inference finished (UTC). NULL if still running.

Table: anomaly_results

Column	Data Type	Constraint	Description
id	UUID	PK, NOT NULL	Unique result identifier
transaction_id	UUID	FK(transactions.id) ON DELETE CASCADE, NOT NULL	Scored transaction
model_run_id	UUID	FK(model_runs.id), NOT NULL	Source analysis run
anomaly_score	NUMERIC(6,4)	NOT NULL, CHECK BETWEEN 0 AND 1	ML anomaly score 0.0000-1.0000
risk_category	VARCHAR(20)	NOT NULL	normal / suspicious / high_risk / critical
shap_values	JSONB	NOT NULL	{feature: shap_value} dict for top features
is_confirmed_anomaly	BOOLEAN	NULL	Manual review override. NULL = unreviewed.
reviewer_notes	TEXT	NULL	Optional annotation from Tax Consultant reviewer
reviewed_at	TIMESTAMP	NULL	When manual review was performed (UTC)

reviewed_by	UUID	NULL, FK(users.id)	User who performed the manual review
--------------------	------	--------------------	--------------------------------------

Table: compliance_checks

Column	Data Type	Constraint	Description
id	UUID	PK, NOT NULL	Unique compliance check identifier
dataset_id	UUID	FK(datasets.id), NOT NULL	Checked dataset
filing_period	VARCHAR(20)	NOT NULL	Filing period covered by this check
compliance_score	NUMERIC(5,2)	NOT NULL, CHECK 0-100	Overall compliance score for the period
total_violations	INTEGER	NOT NULL, DEFAULT 0	Total number of violations detected
critical_count	INTEGER	NOT NULL, DEFAULT 0	Count of critical severity violations
major_count	INTEGER	NOT NULL, DEFAULT 0	Count of major severity violations
minor_count	INTEGER	NOT NULL, DEFAULT 0	Count of minor severity violations
checked_at	TIMESTAMP	NOT NULL, DEFAULT now()	When compliance check was performed (UTC)

Table: violations

Column	Data Type	Constraint	Description
id	UUID	PK, NOT NULL	Unique violation identifier
compliance_check_id	UUID	FK(compliance_checks.id) ON DELETE CASCADE, NOT NULL	Parent compliance check
transaction_id	UUID	NULL, FK(transactions.id)	Affected transaction. NULL for dataset-level violations.
violation_type	VARCHAR(100)	NOT NULL	RULE_CODE: e.g. INVALID_GSTIN, DUPLICATE_INVOICE, MISSING_HSN
severity	VARCHAR(20)	NOT NULL	critical / major / minor
description	TEXT	NOT NULL	Human-readable description of the violation
remediation	TEXT	NOT NULL	Recommended corrective action for the user

Table: reports

Column	Data Type	Constraint	Description
id	UUID	PK, NOT NULL	Unique report identifier
user_id	UUID	FK(users.id), NOT NULL	User who generated the report
dataset_id	UUID	FK(datasets.id), NOT NULL	Source dataset
report_type	VARCHAR(10)	NOT NULL	pdf / excel
file_path	VARCHAR(500)	NOT NULL	S3 key or local path of generated report file
generated_at	TIMESTAMP	NOT NULL, DEFAULT now()	Generation timestamp (UTC)

expires_at	TIMESTAMP	NOT NULL	90 days after generation. Reports deleted by cron after expiry.
status	VARCHAR(20)	NOT NULL, DEFAULT generating	generating / ready / expired / error

Table: audit_logs

Column	Data Type	Constraint	Description
id	BIGSERIAL	PK, NOT NULL	Auto-incrementing audit log ID
user_id	UUID	NULL, FK(users.id)	Acting user. NULL for system-generated events.
action	VARCHAR(100)	NOT NULL	Action code: LOGIN, UPLOAD_DATASET, RUN_ANALYSIS, GENERATE_REPORT, LOGOUT, etc.
resource_type	VARCHAR(50)	NULL	Type of resource acted upon: dataset, report, user, model_run
resource_id	UUID	NULL	ID of the specific resource affected
ip_address	INET	NULL	Client IP address from request headers
details	JSONB	NULL	Additional context: {filename, row_count, model_version, ...}
timestamp	TIMESTAMP	NOT NULL, DEFAULT now()	Event timestamp (UTC). Immutable once written.

4.3 Database Indexes

```
-- Performance indexes
CREATE INDEX idx_transactions_dataset_id ON transactions(dataset_id);
CREATE INDEX idx_transactions_txn_date ON transactions(txn_date);
CREATE INDEX idx_anomaly_results_transaction ON anomaly_results(transaction_id);
CREATE INDEX idx_anomaly_results_model_run ON anomaly_results(model_run_id);
CREATE INDEX idx_anomaly_results_risk ON anomaly_results(risk_category);
CREATE INDEX idx_violations_compliance_check ON violations(compliance_check_id);
CREATE INDEX idx_audit_logs_user_timestamp ON audit_logs(user_id, timestamp DESC);
CREATE INDEX idx_datasets_user_id ON datasets(user_id);
CREATE INDEX idx_reports_user_expires ON reports(user_id, expires_at);
-- Partial index: only index non-normal anomaly results (most queries filter on flagged)
CREATE INDEX idx_flagged_results ON anomaly_results(transaction_id, anomaly_score) WHERE risk_category != 'normal';
-- GIN index on JSONB for SHAP value querying
CREATE INDEX idx_shap_values_gin ON anomaly_results USING GIN(shap_values);
```

5. REST API SPECIFICATION

Base URL: <https://api.karai.io/api/v1> | **Auth:** Bearer JWT in Authorization header | **Content-Type:** application/json (all endpoints)

5.1 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/auth/register	No	Register new user account. Body: {email, password, role}. Returns: {user_id, email, role}
POST	/auth/login	No	Login with credentials. Body: {email, password}. Returns: {access_token, token_type, expires_in}

POST	/auth/logout	Yes	Invalidate current JWT token (adds jti to Redis blacklist). Returns: {message}
POST	/auth/refresh	Yes	Refresh JWT token before expiry. Returns: {access_token, expires_in}
POST	/auth/reset-password	No	Initiate password reset via email OTP. Body: {email}
PUT	/auth/reset-password	No	Complete password reset. Body: {email, otp, new_password}

5.2 Dataset Endpoints

Method	Endpoint	Auth	Description
GET	/datasets	Yes	List all datasets for authenticated user. Query: ?page=1&limit=20&status=ready
POST	/datasets/upload	Yes	Upload CSV file (multipart/form-data). Field: file (CSV), filing_period (string). Returns: {dataset_id, status, row_count}
GET	/datasets/{id}	Yes	Get dataset metadata and ingestion status. Returns: {id, filename, row_count, status, upload_date, filing_period}
DELETE	/datasets/{id}	Yes	Delete dataset and all associated results. Cascades to transactions, anomaly_results, violations.
GET	/datasets/{id}/preview	Yes	Preview first 100 rows of dataset with field mapping. Query: ?mapping={...}

5.3 Analysis Endpoints

Method	Endpoint	Auth	Description
POST	/analysis/run	Yes	Start anomaly detection. Body: {dataset_id, model_name?, contamination?}. Returns: {task_id, model_run_id}
GET	/analysis/{run_id}	Yes	Get model run status and summary metrics. Returns: {status, flagged_count, completed_at, metrics}
GET	/analysis/{run_id}/results	Yes	Paginated anomaly results. Query: ?page&limit;&risk;&category&min;&score. Returns: {results[], total, page}
GET	/analysis/{run_id}/shap/{txn_id}	Yes	SHAP detail for specific transaction. Returns: {shap_values, waterfall_chart_url, top_features}
GET	/analysis/{run_id}/export	Yes	Download full results as Excel. Returns: binary xlsx file

5.4 Compliance Endpoints

Method	Endpoint	Auth	Description
POST	/compliance/check	Yes	Run compliance validation on dataset. Body: {dataset_id}. Returns: {check_id, compliance_score}
GET	/compliance/{check_id}	Yes	Get compliance check summary. Returns: {score, total_violations, by_severity, filing_period}
GET	/compliance/{check_id}/violations	Yes	Paginated violation list. Query: ?severity&page;&limit.; Returns: {violations[], total}

GET	/compliance/history/{dataset_id}	Yes	Compliance score history across all checks for a dataset. Returns: [{filing_period, score, date}]
-----	----------------------------------	-----	---

5.5 Dashboard Endpoints

Method	Endpoint	Auth	Description
GET	/dashboard/summary	Yes	Aggregated KPI cards for authenticated user. Query: ?dataset_id. Returns: {total_txns, flagged, compliance_score, risk_level}
GET	/dashboard/trends	Yes	Time-series data for anomaly count and compliance score. Query: ?dataset_id&from;_date&to;_date&granularity;=monthly
GET	/dashboard/scatter	Yes	Scatter plot data: {transaction_id, amount, anomaly_score, risk_category} for all transactions in dataset
GET	/dashboard/heatmap	Yes	Party x Category anomaly heatmap data. Returns matrix of average anomaly scores per cell.

5.6 Report Endpoints

Method	Endpoint	Auth	Description
POST	/reports/generate	Yes	Queue report generation. Body: {dataset_id, report_type, model_run_id, compliance_check_id?}. Returns: {report_id, status}
GET	/reports	Yes	List all reports for user. Query: ?page&limit;&report_type. Returns: {reports[], total}
GET	/reports/{id}	Yes	Get report metadata and status. Returns: {id, status, generated_at, expires_at, download_url?}
GET	/reports/{id}/download	Yes	Download generated report file. Returns: presigned S3 URL redirect (307) or binary file.
DELETE	/reports/{id}	Yes	Delete report and associated file from storage.

5.7 Admin Endpoints

Method	Endpoint	Auth	Description
GET	/admin/users	Yes	List all users (Admin only). Query: ?page&limit;&role.;
PUT	/admin/users/{id}/role	Yes	Update user role (Admin only). Body: {role}
POST	/admin/users/{id}/lock	Yes	Lock/unlock user account (Admin only). Body: {locked: bool}
GET	/admin/model-runs	Yes	All model runs across all users (Admin only).
POST	/admin/retrain	Yes	Trigger model retraining (Admin only). Body: {dataset_ids[], model_name, parameters}
GET	/admin/metrics	Yes	System health metrics: active users, jobs queued, DB size, avg response time.

5.8 Standard Error Response Schema

```
// All error responses follow this schema: { "status_code": 422, // HTTP status code
"error_code": "VALIDATION_ERROR", // Machine-readable code "message": "Amount must be
positive", // Human-readable message "detail": [ // Optional: field-level validation errors
{"field": "amount", "msg": "ensure this value is greater than 0"} ], "request_id": "req_abc123"
// For log correlation }
```

6. ML PIPELINE ARCHITECTURE

6.1 Feature Engineering — Complete Feature List

Feature Name	Source Column	Transformation	Rationale
amount	amount	Raw (after decryption)	Base amount signal
log_amount	amount	$\text{np.log1p}(\text{amount})$	Handles right-skewed distributions
amount_zscore	amount	StandardScaler per dataset	Standardises scale for IF
day_of_week	txn_date	$\text{date.weekday}() \rightarrow \text{int } 0-6$	Weekend transactions may be anomalous
month	txn_date	$\text{date.month} \rightarrow \text{int } 1-12$	Seasonal patterns in tax behaviour
quarter	txn_date	$(\text{month}-1)//3 + 1 \rightarrow \text{int } 1-4$	GST filing is quarterly
is_weekend	txn_date	day_of_week in [5,6] $\rightarrow$ bool	B2B transactions rarely on weekends
party_txn_count	party_name	GroupBy count per dataset	Low-count parties may be shell entities
party_amount_mean	party_name	GroupBy mean of amounts	Baseline for party-specific anomaly
party_amount_std	party_name	GroupBy std of amounts	Party payment variance
amount_deviation_from_party	amount, party	$(\text{amount} - \text{party_mean}) / (\text{party_std} + \text{eps})$	Z-score relative to party history
category_frequency	category	Value count fraction per dataset	Rare categories may signal misclassification
gstn_valid	gstn	Regex check $\rightarrow$ bool 0/1	Invalid GSTIN is a hard compliance signal
invoice_duplicate_flag	invoice_number	Hash-based duplicate detection $\rightarrow$ bool	Duplicate invoices = GST fraud signal
round_amount_flag	amount	$\text{amount} \% 1000 == 0 \rightarrow \text{bool}$	Suspiciously round amounts flag laundering
amount_percentile_in_dataset	amount	$\text{scipy.stats.percentileofscore}()$	Relative position in overall distribution

6.2 Isolation Forest — Design Rationale and Configuration

Algorithm Choice	Isolation Forest chosen as primary unsupervised detector because: (1) no labelled training data required, (2) linear time complexity $O(n \log n)$, (3) effective in high-dimensional spaces, (4) native anomaly scoring (not binary classification).
n_estimators	200 trees. More trees = more stable score estimates. 100 is default; 200 chosen for financial data precision.
contamination	0.05 (5%) default. Configurable via Settings (0.01-0.20). Represents expected proportion of fraudulent transactions in the dataset.

max_samples	"auto" = min(256, n_samples). Uses subsample for speed.
random_state	42. Fixed for reproducibility. All model versions must use same seed.
Scoring Normalisation	Raw IF scores are negative (anomalous) to positive (normal). Converted to 0-1 scale: score = 1 - (raw_score + 0.5). Values > 0.5 are anomalous by definition.
Threshold Override	Admin-configurable sensitivity threshold allows moving the Normal/Suspicious boundary from 0.3 to any value 0.1-0.6 without retraining.

6.3 SHAP Integration Design

Explainer Init	shap.TreeExplainer(pipeline[-1], check_additivity=False). Initialised once per model version at engine startup.
Input to SHAP	Feature-engineered DataFrame (after Pipeline.transform()). SHAP expects the feature matrix, not raw data.
Output Processing	shap_values array shape: (n_samples, n_features). For each flagged transaction, extract {feature: shap_value} dict.
Top-5 Extraction	Sorted by shap_value descending. Top 5 stored in anomaly_results.shap_values JSONB. Full values available on-demand via /shap/{txn_id} endpoint.
Waterfall Charts	Generated via matplotlib on the fly: shap.waterfall_plot(shap.Explanation(values, base_values, feature_names)). Saved as PNG bytes, served via presigned URL.
Performance	SHAP computation batched: all flagged transactions in one shap_values() call. For 10K transactions with 5% flagged = 500 SHAP computations. ~2-3 seconds total.

7. DATA FLOW DESIGN

7.1 DFD Level 0 — Context Diagram

External Entity	Data Flow Direction	Data Flow Description
SME User / Tax Consultant	→ KarAI	Transaction CSV files, login credentials, report requests, compliance queries
KarAI	→ SME User / Tax Consultant	Anomaly results, compliance scores, audit reports (PDF/Excel), dashboard visualisations
Tax Authority / CA	→ KarAI	Regulatory rule updates (manual upload by admin)
KarAI	→ Tax Authority / CA	Audit-ready compliance reports
Accounting Software (Tally)	→ KarAI	Transaction data via REST API integration
KarAI	→ Email Server (SMTP)	OTP emails, report-ready notifications, alert emails
MLflow Registry	↔ KarAI	Model artifacts (upload on training, download on inference)

7.2 DFD Level 1 — Internal Data Flows

Process	Input	Output	Data Store Interaction
P1: Authentication	Email + Password	JWT Token / Error	Read: users table. Write: audit_logs, Redis blacklist
P2: Data Ingestion	CSV File	Dataset ID + Validation Report	Write: datasets, transactions tables
P3: Feature Engineering	Raw transaction rows	Feature matrix (16 columns)	Read: transactions table
P4: ML Inference	Feature matrix	Anomaly scores array	Read: MLflow model. Write: model_runs, anomaly_results
P5: SHAP Explanation	Feature matrix + trained model	SHAP values dict per transaction	Write: anomaly_results.shap_values
P6: Compliance Check	Transactions + GST rules	Violations list + compliance score	Write: compliance_checks, violations tables
P7: Dashboard Aggregation	anomaly_results + compliance_checks	KPI data, chart data	Read: anomaly_results, compliance_checks, transactions
P8: Report Generation	Aggregated results + user config	PDF/Excel file bytes	Read: all result tables. Write: reports table, S3

7.3 Key Sequence: Run Anomaly Detection

Step	Actor	Action	Async ?
1	Frontend	User clicks "Analyse Dataset". POST /analysis/run {dataset_id}	No
2	FastAPI Router	Validates JWT, checks dataset ownership, validates request body	No
3	AnomalyEngine	Creates model_run record (status=running) in DB. Returns model_run_id.	No
4	FastAPI Router	Enqueues Celery task. Returns {task_id, model_run_id} to frontend (202 Accepted).	No
5	Frontend	Subscribes to WebSocket /ws/analysis/{model_run_id} for status updates.	Yes
6	Celery Worker	Picks up task. Calls AnomalyEngine.run(dataset_id, model_run_id).	Yes
7	AnomalyEngine	Loads transactions from DB. Runs FeaturePipeline.transform().	Yes
8	AnomalyEngine	Loads IF model from MLflow. Runs isolation_forest.predict() and score_samples().	Yes
9	AnomalyEngine	Runs SHAP on flagged transactions. Classifies risk categories.	Yes
10	AnomalyEngine	Bulk-inserts anomaly_results to DB. Updates model_run status=completed.	Yes
11	Celery Worker	Publishes WebSocket message: {status: completed, flagged_count, risk_summary}.	Yes
12	Frontend	Receives WebSocket update. Redirects user to Results view.	Yes

8. SECURITY ARCHITECTURE

8.1 Authentication and Authorisation Flow

Step	Component	Action
Login Request	Client → FastAPI /auth/login	POST {email, password} over HTTPS/TLS1.3
Password Verify	AuthService	passlib.verify(plain_password, stored_hash). Constant-time comparison.
Lockout Check	AuthService → Redis	Check failed_attempts counter. If >= 5 and not expired → raise 403.
Token Generation	AuthService python-jose →	Create JWT: {sub: user_id, role, jti: uuid4(), exp: now+24h, iat: now}
Token Response	FastAPI → Client	{access_token, token_type: "bearer", expires_in: 86400}
Authenticated Request	Client → FastAPI (any protected route)	Authorization: Bearer header
Token Verification	JWTMiddleware python-jose →	Decode + verify signature. Check exp. Check jti not in Redis blacklist.
Role Check	JWTMiddleware RouteConfig →	Compare token.role against required_roles for endpoint. 403 if insufficient.
Logout	AuthService → Redis	SET jti: TRUE EX . Token cannot be reused.

8.2 Role Permission Matrix

Endpoint Group	SME User	Tax Consultant	Administrator
Upload Dataset	Own datasets only	Own datasets only	All datasets
View Anomaly Results	Own results only	Own + client results	All results
Generate Reports	Own datasets only	Own + client datasets	All datasets
Add Reviewer Annotations	No	Yes	Yes
Adjust ML Thresholds	No	No	Yes
Manage Users	No	No	Yes
Trigger Model Retrain	No	No	Yes
View System Metrics	No	No	Yes

8.3 Data Encryption Strategy

Data in Transit	TLS 1.3 enforced on all client-server and server-to-service connections. TLS 1.0/1.1 disabled. HSTS header enabled.
Data at Rest	Sensitive transaction fields (amount, party_name, gstin) encrypted using Fernet (AES-128-CBC + HMAC-SHA256) before DB insert. Encryption key in ENCRYPTION_KEY env var (Fernet-generated 32-byte key, base64-encoded).
Key Management	Fernet key rotated annually. Old key retained for decryption of legacy data. Key never hardcoded. Managed via AWS Secrets Manager in production.
Password Storage	bcrypt(cost=12). 2^12 = 4096 iterations. Upgrade cost factor automatically when bcrypt version updates.
JWT Signing	HS256 with 256-bit random secret (development). RS256 with RSA-2048 keypair (production) — private key signs, public key verifies.

Report Files	S3 server-side encryption (SSE-S3) on all stored report files. Presigned URLs expire in 1 hour.
--------------	---

8.4 Input Validation and Injection Prevention

SQL Injection	SQLAlchemy ORM exclusively. No raw SQL string concatenation. All user inputs parameterised.
XSS Prevention	React frontend escapes all dynamic content. Backend sanitises all string inputs via Pydantic validators.
CSV Injection	Cells beginning with =, +, -, @ prefixed with single quote before storage to prevent formula injection in Excel exports.
File Upload	MIME type validation (must be text/csv). Magic bytes check. Max size 50MB enforced at API level before processing.
Rate Limiting	Redis-backed sliding window: 100 req/min per user per endpoint group. /auth/login limited to 10 req/min per IP.
CORS Policy	Allowed origins whitelist in CORS_ORIGINS env var. No wildcard (*) in production.

9. DEPLOYMENT ARCHITECTURE

9.1 Docker Compose Services

Service	Container Image	Ports	Purpose
karai-api	python:3.12-slim (custom)	8000:8000	FastAPI + Uvicorn. Main API server. 4 worker processes.
karai-worker	python:3.12-slim (custom)	None	Celery worker for async ML jobs. Consumes from karai-queue.
karai-frontend	nginx:alpine (custom)	80:80	Serves React SPA static files. Proxies /api/* to karai-api.
karai-db	postgres:16-alpine	5432:5432	PostgreSQL database. Volume: karai-pgdata.
karai-redis	redis:7-alpine	6379:6379	Redis for Celery broker, rate limiting, JWT blacklist.
karai-beat	python:3.12-slim (custom)	None	Celery Beat scheduler for cron jobs (backups, report cleanup).

9.2 Environment Variables — Complete Reference

```
# Database
DATABASE_URL=postgresql+asyncpg://karai:password@karai-db:5432/karai DATABASE_POOL_SIZE=10
DATABASE_MAX_OVERFLOW=20 # Redis
REDIS_URL=redis://karai-redis:6379/0 # Security
JWT_SECRET_KEY=<256-bit random hex string> JWT_ALGORITHM=HS256 JWT_EXPIRY_HOURS=24
ENCRYPTION_KEY= # ML / MLflow
MLFLOW_TRACKING_URI=http://mlflow:5000
MLFLOW_MODEL_URI=models:/KarAI-IsolationForest/Production DEFAULT_CONTAMINATION=0.05 # AWS
AWS_ACCESS_KEY_ID= AWS_SECRET_ACCESS_KEY= AWS_REGION=ap-south-1
S3_BUCKET_REPORTS=karai-reports-prod S3_PRESIGN_EXPIRY_SECONDS=3600 # Email
SMTP_HOST=email-smtp.ap-south-1.amazonaws.com SMTP_PORT=587 SMTP_USER= SMTP_PASSWORD=
FROM_EMAIL=no-reply@karai.io # App
APP_ENV=production CORS_ORIGINS=https://app.karai.io DOCS_ENABLED=false LOG_LEVEL=INFO
RATE_LIMIT_PER_MINUTE=100
```

9.3 AWS Production Topology

AWS Service	Configuration	Purpose
EC2 t3.large	2 vCPU, 8GB RAM. Ubuntu 22.04. Docker Compose.	Runs API + Worker + Redis + Frontend containers
RDS PostgreSQL	db.t3.medium, Multi-AZ standby, 100GB gp3 storage, automated backups 7 days.	Managed PostgreSQL database
ElastiCache Redis	cache.t3.micro, single-AZ for dev; Multi-AZ for production.	Managed Redis for rate limiting and Celery broker
S3	ap-south-1 region, versioning enabled, lifecycle: delete after 90 days.	Report file storage
CloudFront	Distribution pointing to S3 (frontend assets) and ALB (API). HTTPS only.	CDN for frontend. Reduces latency.
ALB	Application Load Balancer. HTTPS:443 → HTTP:8000 target. Health check: /health.	Load balancing + SSL termination
ACM	SSL certificate for app.karai.io. Auto-renewed.	SSL certificate management
Route 53	A record: app.karai.io → CloudFront. api.karai.io → ALB.	DNS management
SES	ap-south-1. Verified domain karai.io. TLS required.	Transactional email delivery
Secrets Manager	Stores JWT_SECRET_KEY, ENCRYPTION_KEY, DB password.	Secure secret storage

10. ERROR HANDLING STRATEGY

10.1 HTTP Status Code Mapping

HTTP Code	Error Code	Scenario	Retry ?
-----------	------------	----------	---------

400	VALIDATION_ERROR	Invalid request body (wrong types, missing fields, constraint violation)	No — fix request
401	INVALID_TOKEN	Missing, expired, or blacklisted JWT token	No — re-login
403	INSUFFICIENT_ROLE	Valid token but role lacks permission for endpoint	No — wrong role
403	ACCOUNT_LOCKED	Account locked due to repeated failed login attempts	After lockout expiry
404	RESOURCE_NOT_FOUND	Dataset, report, or model run ID not found or not owned by user	No
409	DUPLICATE_RESOURCE	Email already registered; dataset with same hash already uploaded	No — use existing
413	FILE_TOO_LARGE	Uploaded CSV exceeds 50MB limit	No — reduce file
422	UNPROCESSABLE_ENTITY	File is not a valid CSV; PDF parsing failure	No — fix file
429	RATE_LIMIT_EXCEEDED	Request rate limit exceeded	Yes — after 60s
500	INTERNAL_ERROR	Unexpected server error. Logged with request_id.	Yes — transient
503	MODEL_UNAVAILABLE	MLflow model not loaded; inference service unavailable	Yes — after delay
504	INFERENCE_TIMEOUT	ML inference exceeded timeout threshold (60s)	Yes — smaller batch

10.2 Retry and Circuit Breaker Strategy

ML Job Retries	Celery task max_retries=3. Exponential backoff: 30s, 120s, 300s. After 3 failures, model_run.status=failed and user notified via email.
DB Connection Retry	SQLAlchemy pool_pre_ping=True checks connection health before use. Failed connections removed from pool and reconnected automatically.
Report Generation	Same Celery retry policy as ML jobs. If S3 upload fails, retry up to 3 times with exponential backoff.
Email Delivery	SES delivery failures logged. Non-critical path — no retry for email. Report still available via dashboard on failure.
Circuit Breaker	If external service (MLflow, S3, SMTP) fails 5+ times in 60s window, circuit opens for 120s. Requests fail fast with 503. Probed every 120s to close.

10.3 Logging Strategy

Library	structlog 24.1+ for structured JSON logging. Human-readable in development, JSON in production.
Log Levels	DEBUG: development only. INFO: request start/end, business events. WARNING: validation failures, rate limit hits. ERROR: exceptions. CRITICAL: system failures.
Required Fields	Every log entry includes: timestamp (ISO8601 UTC), level, request_id, user_id (if authenticated), endpoint, duration_ms.
Sensitive Data	Passwords, tokens, and encrypted field values NEVER logged. Amount and party_name masked in logs as .
Log Shipping	Production logs shipped to CloudWatch Logs via the awslogs Docker log driver. Retention: 30 days.